Document number: P2047R7
Audience: LEWG
Pablo Halpern
Nina Dinka Ranns
Ville Voutilainen
2024-02-12

An allocator-aware optional type

Abstract

Library types that can potentially hold *allocator-aware (AA)* objects should, themselves, be allocator-aware. A PMR container, for example, depends on AA types following a known AA protocol so that it (the container) can uniformly manage its memory. Even types that don't manage their own memory, such as `tuple`, follow the AA rules when they hold one more more AA elements. (A special case is `pair`, which is not actually AA but effectively follows the rules through special handling in *uses-allocator construction*.)

The current definition of `std::optional` does not follow the rules for an AA type, even when holding an AA value. This limitation makes `std::optional` unsuitable for storage in an AA container when memory allocation customization is needed.

In this paper, we propose a new, allocator-aware `std::basic_optional` usable as a container element type of allocator aware containers, and in any other context where allocator propagation is expected. This new type would not replace the current `std::optional` as the desired behaviour is not compatible with how `std::optional` handles allocator aware types at the moment. We do propose having a special treatement for `std::basic_optional` and `std::optional` that allows for certain implicit conversions between the two types. We also propose an `std::pmr::optional` type which is a specialisation of `std::basic_optional` for `std::pmr::polymorphic_allocator<>`.

This is a complete proposal with formal wording.

change history :
- replaced `alloc_optional` with additional overloads of `make_optional` that take an `allocator_arg_t` : it was observed that `alloc_optional` doesn't actually do any allocation, and that the name is not appropriate
- fixed the order of arguments in the *Returns* clause of what used to be `alloc_optional`
- adjusted for LWG 2833
- added discussion on deduction guides
- removed mentions of using a type alias since the type alias approach has proven to be problematic
- modified the specification of std::pmr::optional to cover both AA and non AA types
- added CTAD specification
- modified the specification of `std::pmr::optional` to be expressed in terms of `std::pmr::polymorphic_allocator<>`, as opposed to `std::pmr::memory_resource`

- moved `std::pmr::optional` to `optional` header as discussed in the reflector discussion
- added request for discussion on making `std::pmr::optional` a more allocator generic type.
- added free swap function to be in line with P0178
R3:
- replaced instances of *M_alloc* with *alloc*
- clarified pmr::optional constructor descriptions to call out uses-allocator construction
- expanded `basic_optional` discussion
R4:
- switched to `std::pmr::optional` being a specialisation of `std::basic_optional`
- modified the wording to cover `std::basic_optional`
R5:

- various clean up of proposed wording
- setting the default allocator to be `remove_cv_t` version of value_type
- removed make facilities
R6:
- cleaning up html format
- clean up of proposed wording
R7:
- added design considerations for special member functions and swap
- improved motivation and summary

Motivation

General Motivation for allocator-aware types

Note: The text below is borrowed nearly verbetim from P3002, which proposes a general policy for when types should use allocators

Memory management is a major part of building software. Numerous facilities in the C++ Standard library exist to give the programmer maximum control over how their program uses memory:

- `std::unique_ptr` and `std::shared_ptr` are parameterized with *deleter* objects that control, among other things, how memory resources are reclaimed.
- `std::vector` is preferred over other containers in many cases because its use of contiguous memory provides optimal cache locality and minimizes allocate/deallocate operations. Indeed, the LEWG has spent a lot of time on `flat_set` (P1222R0) and `flat_map` (P0429R3), whose underlying structure defaults to `vector` for this reason.
- Operators `new` and `delete` are replaceable, giving programmers global control over how memory is allocated.
- The C++ object model makes a clear distinction between an object's memory footprint and it's lifetime.
- Language constructs such as `void*` and `reinterpet_cast` provide fine-grained access to objects' underlying memory.
- Standard containers and strings are parameterized with allocators, providing object-level control over memory allocation and element construction.

This fine-grained control over memory that C++ gives the programmer is a large part of why C++ is applicable to so many domains — from embedded systems with limited memory budgets to games, high-frequency trading, and scientific simulations that require cache locality, thread affinity, and other memory-related performance optimizations.

An in-depth description of the value proposition for allocator-aware software can be found in P2035R0. Standard containers are the most ubiquitous examples of *allocator-aware* types. Their `allocator_type` and `get_allocator` members and allocator-parameterized constructors allow them to be used like Lego® parts that can be combined and nested as necessary while retaining full programmer control over how the whole assembly allocates memory. For *scoped allocators* — those that apply not only to the top-level container, but also to its elements — having each element of a container support a predictable allocator-aware interface is crucial to giving the programmer the ability to allocate all memory from a single memory resource, such as an arena or pool. Note that the allocator is a *configuration* parameter of an object and does not contribute to its value.

In short, the principles underlying this policy proposal are:

1. The Standard Library should be general and flexible. To the extent possible, the user of a library class should have control over how memory is allocated.

2. The Standard Library should be consistent. The use of allocators should be consistent with the existing allocator-aware classes and class templates, especially those in the containers library.

3. The parts of the Standard Library should work together. If one part of the library gives the user control over memory allocation but another part does not, then the second part undermines the utility of the first.

4. The Standard Library should encapsulate complexity. Fully general application of allocators is potentially complex and is best left to the experts implementing the Standard Library. Users can choose their own subset of desirable allocator behavior only if the underlying Library classes allow them to choose their preferred approach, whether it be stateless allocators, statically typed allocators, polymorphic allocators, or no allocators.

Motivation for an Allocator-aware `optional`

Although the engaged object in an `std::optional` can be initialized with any set of valid constructor arguments, including allocator arguments, the fact that the `optional` itself is not allocator-aware prevents it from working consistently with other parts of the standard library, specifically those parts that depend on *uses-allocator construction* (section [allocator.uses.construction]) in the standard). For example:

```
pmr::monotonic_buffer_resource rsrc;
pmr::polymorphic_allocator<> alloc{ &rsrc };
using Opt = optional<pmr::string>;
Opt o = make_obj_using_allocator<Opt>(alloc, in_place, "hello");
assert(o->get_allocator() == alloc);  // FAILS
```

Even though an allocator is supplied, it is not used to construct the `pmr::string` within the resulting `optional` object because `optional` does not have the necessary hooks for `make_obj_using_allocator` to recognize it as being allocator-aware. Note that, although this example and the ones that follow use `pmr::polymorphic_allocator`, the same issues would apply to any scoped allocator.

Uses-allocator construction is rarely used directly in user code. Instead, it is used within the implementation of standard containers and scoped allocators to ensure that the allocator used to construct the container is also used to construct its elements. Continuing the example above, consider what happens if an `optional` is stored in a `pmr::vector`, compared to storing a truly allocator-aware type (`pmr::string`):

```
pmr::vector<pmr::string> vs(alloc);
pmr::vector<Opt>         vo(alloc);

vs.emplace_back("hello");
vo.emplace_back("hello");

assert(vs.back().get_allocator() == alloc);   // OK
assert(vo.back()->get_allocator() == alloc);  // FAILS
```

An important invariant when using a scoped allocator such as `pmr::polymorphic_allocator` is that the same allocator is used throughout an object hierarchy. It is impossible to ensure that this invariant is preserved when using `std::optional`, even if the element is originally inserted with the correct allocator, because `optional` does not remember the allocator used to construct it and cannot therefore re-instate the allocator going from disengaged to engaged:

```
vo.emplace_back(in_place, "hello", alloc);
assert(vo.back()->get_allocator() == alloc);  // OK

vo.back() = nullopt;    // Disengage
vo.back() = "goodbye";  // Re-engage
assert(vo.back()->get_allocator() == alloc);  // FAILS
```

Finally, when using assignment, the value stored in the `optional` is set sometimes by construction and other times by assignment. Depending on the allocator type's propagation traits, it is difficult to reason about the resulting allocator:

```
Opt o1{nullopt};    // Disengaged -- does not use an allocator
Opt o2{ "hello" };  // String uses a default-constructed allocator
o1 = pmr::string("goodbye", alloc);    // Constructs the string
o2 = pmr::string("goodbye", alloc);    // Assigns to the string
assert(o1->get_allocator() == alloc);  // OK, set by move construction
assert(o2->get_allocator() == alloc);  // ERROR, set by assignment
```

Summary of the proposed feature

This paper proposes an allocator-aware `optional`. Unfortunately, it would be complicated to add an allocator to the current `std::optional` without causing API and ABI compatibility issues and/or imposing long compile times on code that does not benefit from the change. For this reason, we are proposing a new class template, `basic_optional`, which works like `optional`, but adds allocator support.

The key attributes of `basic_optional` that make it different from `optional` are:

- It has an optional `Allocator` template parameter. If not supplied, `Allocator` is determined from `T::allocator_type` if that exists, otherwise `std::allocator<byte>`.
- It has a public `allocator_type` member to advertise that it is allocator-aware.
- There are allocator-enabled versions of every constructor.
- The supplied allocator is forwarded to the constructor of the engaged object, if that object uses a compatible allocator type. The allocator is retained so that it can be supplied to the contained object whenever the `basic_optional` goes from disengaged to engaged. Allocator traits, including propagation traits, are respected.
- It has an alias, `pmr::optional`, for which `Allocator` is hard-coded to `pmr::polymorphic_allocator<>`.

Design decisions

Default constructor considerations

Default constructor always does uses_allocator construction in case the default allocator construction produces different allocators each time. Possible optimisation opportunity for allocators which have is_always_equal set to true, but no benefit in triviality or constexpr - uses_allocator construction will be constexpr if it can be constexpr either way.

No conditional triviality of the default constructor - the internal state needs to be initialised. This is the same as in std::optional.

Possible additional consideration is for basic_optional where the value type is not allocator aware - such a basic_variant constexpr considerations would only depend on the constexpr construction of the 0th alternative. We do not propose standardising such optimisations.

Copy constructor considerations

Copy construction does not do uses_allocator construction - the value type is expected to behave according to the traits and get the right allocator. This allows for triviality if allocator is always equal and the constructed type can be trivially copy constructed. If allocator is not always equal, we need to check the traits to get the right allocator and that initialisation can not be trivial. Possible additional consideration is for basic_optional where the value type is not allocator aware - such a basic_optional does not need to store the allocator and triviality only depends on the triviality of the value type. This proposal does not consider such optimisation.

Move constructor considerations

Move construction does not do uses_allocator construction - the value type is expected to behave according to the traits and get the right allocator. This allows for triviality if allocator can be trivially copy constructed and the constructed type can be trivially move constructed. Possible additional consideration is for basic_optional where the value type is not allocator aware - such a basic_optional does not need to store the allocator and triviality only depends on the triviality of the value type. This proposal does not consider such optimisation.

Value constructor considerations

Value construction must do uses_allocator construction if allocators aren't always equal- the plain copy/move construction may use the wrong allocator.

The paper does not propose exception specification on this value constructor and considers it QOI

Exception specification considerations :
- for non AA type, the exception specification is equivalent to is_nothrow_constructible_v<ValueType, T>
- for AA type, to get a possibly no throw construction, we need to delegate to plain copy/move which can possibly reuse the allocation. The delegation can only happen if allocators are always equal. If this constructor checked for allocators being always equal before deciding whether to do uses allocator construction or not, it would make sense for all inplace constructors and allocator extended constructors to do the same. This paper does not propose such considerations.

Copy assignment constructor considerations

First the allocator is possibly modified based on the traits, then the assignment/construction is done as normal. Triviality can only happen if allocator is always equal as any other situation possibly requires a modification in the allocator that can not be checked at compiler time (the resulting allocator in copy construction can not be checked against the POCCA trait) Exception can be thrown by the assignment or the copy construction. The assignment never has to use the possibly modified allocator as the type is expected to follow the allocator traits. The construction doesn't need to use the allocator if allocator is always equal or if the value type is not allocator aware. In all other cases, the copy construction needs to explicitly provide the allocator.

With the introduction of allocators, we now have to consider when we can do plain copy construction and when allocator extended construction is needed.
If type is non AA, we can do plain copy construction.
If the type is AA and allocator is always equal, we can also do plain copy construction. Note that, for allocator is always equal case, plain copy construction and allocator copy construction will effectively result in the same allocator. We do not consider the possibility of doing non allocator extended copy construction for the case where the copy assignment allocator propagation matches the resulting allocator at copy construction as that case can't be checked at compile time. In all other cases, copy construction must use allocator extended copy construction to guarantee that the right allocator is used for the value_type.

Move assignment constructor considerations

First the allocator is possibly modified based on the traits, then the assignment/construction is done as normal.
In addition to usual triviality requirements, triviality can only happen if the allocator is propagated for move assignment(POCMA=true) or if the allocator is always equal.
Possible additional consideration is for basic_optional where the value type is not allocator aware - such a basic_optional does not need to store the allocator and the triviality only depends on the triviality of the value type operations. This proposal does not consider such optimisation.

The exception specification needs to consider assignment and move construction operations. Move assignment can be done without explicilty using the allocator even for AA types. - an AA type is expected to follow the allocator traits. Move construction can be done directly for AA types if the allocator is always equal or if POCMA=true. Otherwise, the move construction needs to explicitly specify the allocator

Value assignment constructor considerations

Value construction must do uses_allocator construction if allocators aren't always equal- the plain copy/move construction may use the wrong allocator. Exception specification :
- for non AA type, the exception specification is equivalent to is_nothrow_assignable_v<Tj, T> & & is_nothrow_constructible_v<Tj, T>
- for AA type, any allocation is potentially throwing. To get a possibly no throw construction, we need to delegate to plain move which can possibly reuse the allocation. Such delegation can only happen if allocators are always equal.
Possible additional consideration is for basic_variant where no types are allocator aware - such a basic_variant does not need to ever allocate and exception specification depends only on the alternatives This proposal does not consider such optimisation.

Swap considerations

First the allocator is possibly modified based on the traits, then the swap/construction is done as normal. Exception specification : To get a possibly no throw construction, we need to delegate to plain move which can possibly reuse the allocation. The delegation can only happen if allocators are always equal or if propagate_on_container_swap == true (for such a type, the resulting allocator in move construction is the required allocator as move construction always propagates the allocator.) This means that swap can only be noexcept if
- value type is is_nothrow_move_constructible and is_nothrow_constructible_v, and
- allocator is always equal or propagate_on_container_swap == true

`basic_optional` supports non-scoped propagating allocators

There are two ways of viewing `basic_optional<T>` from allocator propagation perspective :
#1 `basic_optional<T>` is like a `std::tuple`, i.e. it only accepts the allocator at construction so it can forward it to the value_type object. One can use a non-scoped propagation allocator, and when using a scoped propagation allocator `basic_optional<T>` will not "consume" an allocator level. An optional object is in a way like a tuple object as it does not use the allocator itself, it only passes it into the value_type object.
#2 `basic_optional<T>` is like a `std::vector`, i.e. it is a container of one or zero elements, and one should use a scoped propagating allocator if one wants the value_type object to use the allocator. In this approach `basic_optional<T>` will "consume" an allocator level. Using non-scoped propagating allocators makes little sense in this scenario.

The proposal implements #1 as `basic_optional` itself does not allocate any memory so it makes little sense for it to consume an alloctor level.

The basic design of an AA optional is straight-forward: Add an allocator to all of its constructors and use that allocator to construct the value object each type the optional is engaged. However, when holding a non-AA type, there is no reason to pay the overhead of storing an allocator.

`uses_allocator<basic_optional<T,Alloc>>` corresponds to `uses_allocator<T,Alloc>`. We believe there is no need to support the AA constructor interface for non-AA types. Generic programming should use uses-allocator construction and `std::make_obj_using_allocator` to invoke the correct constructors.

Conversions between `std::optional<T>` and `std::basic_optional<T,Alloc>`

Consider:

```
basic_optional<T,Alloc> x;
optional<T> y = x; // #1
basic_optional<T,Alloc> z = y; // #2
```

```
optional<T> foo_constref(const optional<T>& );
foo_constref(x); // #3

void foo_ref(optional<T>&);
foo_ref(x) // #4
```

In the example above, we do not believe #1,#2, and #3 are ever problematic, but may be useful for code which currently uses `optional`. However, #4, if allowed, would potentially modify the `basic_optional` in a way that does not preserve the allocator requirements. Note that #4 is only problematic if `uses_allocator<basic_optional<T,Alloc>> == true`.

Allowing #4 for cases where `uses_allocator<basic_optional<T,Alloc>> == false` would make re-using codebases which traffic in non allocator aware `optional` possible when allocator does not matter. However, it adds to the complexity of design. It is also questionable whether there is a need for this conversion. One can have two reasons to use `basic_optional` with non allocator aware types:
- writing generic code which serves both allocator and non allocator aware types. Allowing interoperability with `optional` for only certain cases seems unhelpful in such a case.
- using `basic_optional` for all optional types for simplicity purposes. Allowing interoperability with `optional` would be useful in this case.
We propose to not implement conversion #4 until the time it is needed. However, library implementors might want to consider this as a possible extension because it might inform the implementation design they go for.

Allocator used in value_or

It is not all that obvious which allocator should be used for the object returned by `value_or`. Should the decision be left to the type or should it be mandated by the optional ? That is, should the object be constructed by plain copy/move construction or with uses-allocator construction? The proposal leaves the decision to the value_type. If the user cares about the allocator of the returned object, it should be explicitly provided by the user. We may consider providing allocator extended version of `value_or` in the future, if this use case proves to be common enough.

Non allocator aware `basic_optional` is not an alias for `optional`

An early draft of this proposal suggested using a type alias where a non-AA `basic_optional` aliases `std::optional`, and AA `basic_optional` aliases a new unnamed type. However, this causes usability issues. Type deduction in cases like :

```
template <typename T>
void foo(basic_optional<T>);
```

did not work. The above was equivalent to

```
template <typename T, typename Alloc>
void foo(std::conditional<std::uses_allocator <T, Alloc<>>::value,
__basic_optional<T>,
std::optional<T>>:type);
```

and using the nested `type` of the `std::conditional` as the function template parameter made for an undeduced context.

Make facilities are not provided

Make facilities have been deemed unnecessary with the availability of CTAD. The current version of the paper doesn't propose `std::make_basic_optional` nor `std::pmr::make_optional`. Previous version suggested those be included, but with implementation experience, we found a complexity in allowing the allocator type to be both specifiable and defaulted in non allocator extended version of the make facility. We deem such complexity unnecessary with the advent of the CTAD feature.

Allocator aware types are assumed to be move enabled

`basic_optional` uses explicit allocator construction except in cases where invoking a direct `value_type` operation allows for move operations to remain noexcept. This is the case in move constructor for all allocators, and in move assignment for allocators that have `propagate_on_container_move_assignment==true`. If the `value_type` is allocator aware, but does not support move semantics (i.e. moves deteriorate to copies), it is possible that the allocator of the `value_type` object will get out of sync with the allocator of the `basic_optional`. We do not expect such types to exist.

Proposed wording

Modify 22.5.2 Header <optional> synopsis

```
namespace std {
 // [optional.optional], class template optional
 template<class T>
   class optional;                                  // partially freestanding
 // [optional.basic.optional], class template basic_optional
 template<class T, class Allocator>
   class basic_optional;                            // partially freestanding
…
 // [optional.relops], relational operators
 template<class T, class U>
   constexpr bool operator==(const optional<T>&, const optional<U>&);
 template<class T, class U>
   constexpr bool operator!=(const optional<T>&, const optional<U>&);
 template<class T, class U>
   constexpr bool operator<(const optional<T>&, const optional<U>&);
 template<class T, class U>
   constexpr bool operator>(const optional<T>&, const optional<U>&);
 template<class T, class U>
   constexpr bool operator<=(const optional<T>&, const optional<U>&);
 template<class T, class U>
   constexpr bool operator>=(const optional<T>&, const optional<U>&);
 template<class T, three_way_comparable_with<T> U>
   constexpr compare_three_way_result_t<T, U>
     operator<=>(const optional<T>&, const optional<U>&);
 template<class T, class AllocatorT, class U, class AllocatorU>
   constexpr bool operator==(const basic_optional<T, AllocatorT>&, const basic_optional<U, AllocatorU>&);
 template<class T, class AllocatorT, class U, class AllocatorU>
   constexpr bool operator!=(const basic_optional<T, AllocatorT>&, const basic_optional<U, AllocatorU>&);
 template<class T, class AllocatorT, class U, class AllocatorU>
   constexpr bool operator<(const basic_optional<T, AllocatorT>&, const basic_optional<U, AllocatorU>&);
 template<class T, class AllocatorT, class U, class AllocatorU>
   constexpr bool operator>(const basic_optional<T, AllocatorT>&, const basic_optional<U, AllocatorU>&);
 template<class T, class AllocatorT, class U, class AllocatorU>
   constexpr bool operator<=(const basic_optional<T, AllocatorT>&, const basic_optional<U, AllocatorU>&);
 template<class T, class AllocatorT, class U, class AllocatorU>
   constexpr bool operator>=(const basic_optional<T, AllocatorT>&, const basic_optional<U, AllocatorU>&);
 template<class T, three_way_comparable_with<T> U>
   constexpr compare_three_way_result_t<T, U>
     operator<=>(const basic_optional<T, AllocatorT>&, const basic_optional<U, AllocatorU>&);
```

```
// [optional.nullops], comparison with nullopt
template<class T> constexpr bool operator==(const optional<T>&, nullopt_t) noexcept;
template<class T>
  constexpr strong_ordering operator<=>(const optional<T>&, nullopt_t) noexcept;
template<class T, class Allocator> constexpr bool operator==(const basic_optional<T, Allocator>&, nullopt_t) noexcept;
template<class T, class Allocator>
  constexpr strong_ordering operator<=>(const basic_optional<T, Allocator>&, nullopt_t) noexcept;

// [optional.comp.with.t], comparison with T
template<class T, class U> constexpr bool operator==(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator==(const T&, const optional<U>&);
template<class T, class U> constexpr bool operator!=(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator!=(const T&, const optional<U>&);
template<class T, class U> constexpr bool operator<(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator<(const T&, const optional<U>&);
template<class T, class U> constexpr bool operator>(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator>(const T&, const optional<U>&);
template<class T, class U> constexpr bool operator<=(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator<=(const T&, const optional<U>&);
template<class T, class U> constexpr bool operator>=(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator>=(const T&, const optional<U>&);
template<class T, class U>
    requires (!is-derived-from-optional<U>) && three_way_comparable_with<T, U>
  constexpr compare_three_way_result_t<T, U>
    operator<=>(const optional<T>&, const U&);
template<class T, class AllocatorT, class U> constexpr bool operator==(const basic_optional<T, AllocatorT>&, const U&);
template<class T, class U, class AllocatorU> constexpr bool operator==(const T&, const basic_optional<U,AllocatorU>&);
template<class T, class AllocatorT, class U> constexpr bool operator!=(const basic_optional<T, AllocatorT>&, const U&);
template<class T, class U, class AllocatorU> constexpr bool operator!=(const T&, const basic_optional<U,AllocatorU>&);
template<class T, class AllocatorT, class U> constexpr bool operator<(const basic_optional<T, AllocatorT>&, const U&);
template<class T, class U, class AllocatorU> constexpr bool operator<(const T&, const basic_optional<U,AllocatorU>&);
template<class T, class AllocatorT, class U> constexpr bool operator>(const basic_optional<T, AllocatorT>&, const U&);
template<class T, class U, class AllocatorU> constexpr bool operator>(const T&, const basic_optional<U,AllocatorU>&);
template<class T, class AllocatorT, class U> constexpr bool operator<=(const basic_optional<T, AllocatorT>&, const U&);
template<class T, class U, class AllocatorU> constexpr bool operator<=(const T&, const basic_optional<U,AllocatorU>&);
template<class T, class AllocatorT, class U> constexpr bool operator>=(const basic_optional<T, AllocatorT>&, const U&);
template<class T, class U, class AllocatorU> constexpr bool operator>=(const T&, const basic_optional<U,AllocatorU>&);
template<class T, class AllocatorT, class U>
    requires (!is-derived-from-optional<U>) && three_way_comparable_with<T, U>
  constexpr compare_three_way_result_t<T, U>
    operator<=>(const basic_optional<T, AllocatorT>&, const U&);

// [optional.specalg], specialized algorithms
template<class T>
  constexpr void swap(optional<T>&, optional<T>&) noexcept(see below);

template<class T, class Allocator>
  constexpr void swap(basic_optional<T, Allocator>&, basic_optional<T, Allocator>&) noexcept(see below);

template<class T>
  constexpr optional<see below> make_optional(T&&);
template<class T, class... Args>
  constexpr optional<T> make_optional(Args&&... args);
template<class T, class U, class... Args>
  constexpr optional<T> make_optional(initializer_list<U> il, Args&&... args);

// [optional.hash], hash support
template<class T> struct hash;
template<class T> struct hash<optional<T>>;
template<class T,  class Allocator> struct hash<basic_optional<T, Allocator>>;

namespace pmr {
      template<class T>
    using optional = basic_optional<T, polymorphic_allocator<>>;
}
```

}

Add new section after 22.5.3

Class template basic_optional [basic.optional]

22.5.x.1 General [basic.optional.general]

```
namespace std {
 template<class T, class Allocator = see below>
 class basic_optional {
 public:
  using value_type = T;
  using allocator_type = Allocator;

  // [basic.optional.ctor], constructors
  constexpr basic_optional() noexcept;
  constexpr basic_optional(nullopt_t) noexcept;
  constexpr basic_optional(const basic_optional&);
  constexpr basic_optional(basic_optional&&) noexcept(see below);
  template<class... Args>
   constexpr explicit basic_optional(in_place_t, Args&&...);
  template<class U, class... Args>
   constexpr explicit basic_optional(in_place_t, initializer_list<U>, Args&&...);
  template<class U = T>
   constexpr explicit(see below) basic_optional(U&&);
 template<class AllocatorU>
   constexpr basic_optional(const basic_optional <T, AllocatorU>&);
  template<class AllocatorU>
   constexpr basic_optional(basic_optional<T, AllocatorU>&&) noexcept(see below);
 template<class U , class AllocatorU>
   constexpr explicit(see below) basic_optional(const basic_optional <U, AllocatorU>&);
  template<class U , class AllocatorU>
   constexpr explicit(see below) basic_optional(basic_optional<U, AllocatorU>&&);
   constexpr explicit(see below) basic_optional(const optional <T>&);
   constexpr explicit(see below) basic_optional(optional<T>&&);
  template<class U>
   constexpr explicit(see below) basic_optional(const optional <U>&);
  template<class U>
   constexpr explicit(see below) basic_optional(optional<U>&&);

// allocator-extended constructors
  constexpr basic_optional(allocator_arg_t, const allocator_type& a) noexcept;
  constexpr basic_optional(allocator_arg_t, const allocator_type& a, nullopt_t) noexcept;
  constexpr basic_optional(allocator_arg_t, const allocator_type& a, const basic_optional&);
  constexpr basic_optional(allocator_arg_t, const allocator_type& a, basic_optional&&);
  template<class... Args>
   constexpr basic_optional(allocator_arg_t, const allocator_type& a, in_place_t, Args&&...);
  template<class U, class... Args>
   constexpr basic_optional(allocator_arg_t, const allocator_type& a, in_place_t, initializer_list<U>, Args&&...);
  template<class U = T>
   constexpr basic_optional(allocator_arg_t, const allocator_type& a, U&&);
 template<class AllocatorU>
   constexpr basic_optional(allocator_arg_t, const allocator_type& a, const basic_optional <T, AllocatorU>&);
  template<class AllocatorU>
   constexpr basic_optional(allocator_arg_t, const allocator_type& a, basic_optional<T, AllocatorU>&&);
 template<class U , class AllocatorU>
   constexpr basic_optional(allocator_arg_t, const allocator_type& a, const basic_optional <U, AllocatorU>&);
  template<class U , class AllocatorU>
   constexpr basic_optional(allocator_arg_t, const allocator_type& a, basic_optional<U, AllocatorU>&&);
   constexpr basic_optional(allocator_arg_t, const allocator_type& a, const optional <T>&);
   constexpr basic_optional(allocator_arg_t, const allocator_type& a, optional<T>&&);
  template<class U>
   constexpr basic_optional(allocator_arg_t, const allocator_type& a, const optional <U>&);
  template<class U>
   constexpr basic_optional(allocator_arg_t, const allocator_type& a, optional<U>&&);
```

```
  // [basic.optional.dtor], destructor
  constexpr ~optional();

  // [basic.optional.assign], assignment
  constexpr basic_optional& operator=(nullopt_t) noexcept;
  constexpr basic_optional& operator=(const basic_optional&);
  constexpr basic_optional& operator=(basic_optional&&) noexcept(see below);
  template<class U = T> constexpr basic_optional& operator=(U&&);
  template<class U, class AllocatorU> constexpr basic_optional& operator=(const basic_optional <U, AllocatorU>&);
  template<class U, class AllocatorU> constexpr basic_optional& operator=(basic_optional <U, AllocatorU>&&) noexcept(see below);
  template<class U> constexpr basic_optional& operator=(const optional <U>&);
  template<class U> constexpr basic_optional& operator=(optional <U>&&);
  template<class... Args> constexpr T& emplace(Args&&...);
  template<class U, class... Args> constexpr T& emplace(initializer_list<U>, Args&&...);

  // [basic.optional.swap], swap
  constexpr void swap(basic_optional&) noexcept(see below);


 // [basic.optional.al], allocator
  allocator_type get_allocator() const;

  // [basic.optional.observe], observers
  constexpr const T* operator->() const noexcept;
  constexpr T* operator->() noexcept;
  constexpr const T& operator*() const & noexcept;
  constexpr T& operator*() & noexcept;
  constexpr T&& operator*() && noexcept;
  constexpr const T&& operator*() const && noexcept;
  constexpr explicit operator bool() const noexcept;
  constexpr bool has_value() const noexcept;
  constexpr const T& value() const &;                 // freestanding-deleted
  constexpr T& value() &;                             // freestanding-deleted
  constexpr T&& value() &&;                           // freestanding-deleted
  constexpr const T&& value() const &&;               // freestanding-deleted
  template<class U> constexpr T value_or(U&&) const &;
  template<class U> constexpr T value_or(U&&) &&;

// [basic_optional.monadic], monadic operations
  template<class F> constexpr auto and_then(F&& f) &;
  template<class F> constexpr auto and_then(F&& f) &&;
  template<class F> constexpr auto and_then(F&& f) const &;
  template<class F> constexpr auto and_then(F&& f) const &&;
  template<class F> constexpr auto transform(F&& f) &;
  template<class F> constexpr auto transform(F&& f) &&;
  template<class F> constexpr auto transform(F&& f) const &;
  template<class F> constexpr auto transform(F&& f) const &&;
  template<class F> constexpr optional or_else(F&& f) &&;
  template<class F> constexpr optional or_else(F&& f) const &;

  // [basic.optional.mod], modifiers
  constexpr void reset() noexcept;

 private:
   T *val;       // exposition only
   Allocator_type *alloc;       // exposition only
 };

}
```

Any instance of basic_optional<T, Allocator> at any given time either contains a value or does not contain a value. When an instance of basic_optional<T, Allocator> *contains a value*, it means that an object of type T, referred to as the basic_optional object's *contained value*, is allocated within the storage of the basic_optional object. Implementations are not permitted to use additional storage, such as dynamic memory, to allocate its contained value. When an object of type basic_optional<T, Allocator> is contextually converted to bool, the conversion returns true if the object contains a value; otherwise the conversion returns false.

When a basic_optional<T, Allocator> object contains a value, member val points to the contained value.

T shall be a type other than *cv* in_place_t or *cv* nullopt_t that meets the *Cpp17Destructible* requirements (Table 35).

Template argument Allocator shall satisfy the *Cpp17Allocator* requirements (16.4.4.6.1). A an instance of Allocator is maintained by the basic_optional object during the lifetime of the object or until the allocator is replaced. The allocator instance is set at basic_optional object creation time and may be replaced only via assignment or swap as specified below. The allocator instance is used to pass to uses-allocator construction of a value object as specified below.

Default template argument for the template parameter Allocator is T::allocator_type if T::allocator_type designates a type, and std::allocator<T> otherwise.

22.5.x.2 Constructors [basic.optional.ctor]

The exposition-only variable template *converts-from-any-cvref* is used by some constructors for basic_optional.

```
template<class T, class W>
constexpr bool converts-from-any-cvref =  // exposition only
 disjunction_v<is_constructible<T, W&>, is_convertible<W&, T>,
          is_constructible<T, W>, is_convertible<W, T>,
          is_constructible<T, const W&>, is_convertible<const W&, T>,
          is_constructible<T, const W>, is_convertible<const W, T>>;
```

```
constexpr basic_optional() noexcept;
constexpr basic_optional(nullopt_t) noexcept;
```

Postconditions: *this does not contain a value.

Remarks: alloc is default initialised. No contained value is initialized. If Allocator's default construct is a constexpr construct these constructors are constexpr constructors ([dcl.constexpr]).

```
constexpr basic_optional(const basic_optional& rhs);
```

Effects: If allocator_traits<Allocator>::is_always_equal::value is true, alloc is default-initialized. Otherwise, alloc is direct-initialized with an allocator_traits<Allocator>::select_on_container_copy_construction(rhs.get_allocator()). If rhs contains a value, direct-non-list-initializes the contained value with *rhs

Postconditions: rhs.has_value() == this->has_value().

Throws: Any exception thrown by the selected constructor of T.

Remarks: This constructor is defined as deleted unless is_copy_constructible_v<T> is true. This constructor is trivial if :

- is_trivially_copy_constructible_v<T> is true,
- allocator_traits<Allocator>::is_always_equal::value is true, and
- is_trivially_copy_constructible_v<Allocator> is true

```
constexpr basic_optional(optional&& rhs) basic_optional noexcept(see below);
```

Constraints: is_move_constructible_v<T> is true

Effects: alloc is direct-initialized with w.get_allocator(). If rhs contains a value, direct-non-list-initializes the contained value with std :: move(*rhs). rhs.has_value() is unchanged

Postconditions: rhs.has_value() == this->has_value().

Throws: Any exception thrown by the selected constructor of T.

Remarks: The exception specification is equivalent to is_nothrow_move_constructible_v<T>. This constructor is trivial if :

- is_trivially_move_constructible_v<T> is true, and
- is_trivially_move_constructible_v<Allocator> is true

```
template<class... Args> constexpr explicit basic_optional(in_place_t, Args&&... args);
```

Constraints: is_constructible_v<T, Args...> is true.

Effects: alloc is default-initialized. The contained value is constructed by uses-allocator construction with allocator alloc and std :: forward<Args>(args)...

Postconditions: *this contains a value.

Throws: Any exception thrown by the selected constructor of T.

Remarks: If Allocator's default constructor and T's selected constructor are constexpr constructors, this constructor is a constexpr constructor.

```
template<class U, class... Args>
  constexpr explicit basic_optional(in_place_t, initializer_list<U> il, Args&&... args);
```

Constraints: is_constructible_v<T, initializer_list<U>&, Args...> is true.

Effects: alloc is default-initialized. The contained value is constructed by uses-allocator construction with allocator alloc and il, std :: forward<Args>(args)...
Postconditions: *this contains a value.
Throws: Any exception thrown by the selected constructor of T.
Remarks: If Allocator's default constructor and T's selected constructor are constexpr constructors, this constructor is a constexpr constructor.

template<class U = T> constexpr explicit(*see below*) basic_optional(U&& v);
Constraints:
- is_constructible_v<T, U> is true,,
- is_same_v<remove_cvref_t<U>, in_place_t> is false,
- is_same_v<remove_cvref_t<U>, allocator_arg_t> is false,
- is_same_v<remove_cvref_t<U>, basic_optional> is false, and
- if T is *cv* bool, remove_cvref_t<U> is not a specialization of basic_optional.

Effects: alloc is default-initialized. The contained value is constructed by uses-allocator construction with allocator alloc and std :: forward<U>(v).
Postconditions: *this contains a value.
Throws: Any exception thrown by the selected constructor of T.
Remarks: If Allocator's default constructor and T's selected constructor are constexpr constructors, this constructor is a constexpr constructor.

template<class AllocatorU>
constexpr basic_optional(const basic_optional <T, AllocatorU>&);
Constraints:
- is_copy_constructible_v<T> is true.

Effects: If allocator_traits<Allocator>::is_always_equal::value is true, alloc is default-initialized. Otherwise, alloc is direct-initialized with an allocator_traits<Allocator>::select_on_container_copy_construction(rhs.get_allocator()). If rhs contains a value, direct-non-list-initializes the contained value with *rhs.
Postconditions: rhs.has_value() == this->has_value().
Throws: Any exception thrown by the selected constructor of T.
Remarks: If is_convertible_v<const AllocatorU&, Allocator> is false, this constructor is defined as deleted. If Allocator's default constructor and T's selected constructor are constexpr constructors, this constructor is a constexpr constructor.

template<class U> constexpr basic_optional(optional<T, AllocatorU>&& rhs) noexcept(*see below*);
Constraints:
- is_move_constructible_v<T> is true.

Effects: alloc is direct-initialized with rhs.get_allocator(). If rhs contains a value, direct-non-list-initializes the contained value with *rhs
Postconditions: rhs.has_value() == this->has_value().
Throws: Any exception thrown by the selected constructor of T.
Remarks: If is_convertible_v<const AllocatorU&, Allocator> is false, this constructor is defined as deleted. The exception specification is equivalent to is_nothrow_move_constructible_v<T>. If Allocator's default constructor and T's selected constructor are constexpr constructors, this constructor is a constexpr constructor.

template<class U , class AllocatorU>
constexpr explicit(*see below*) basic_optional(const basic_optional <U, AllocatorU>&);
Constraints:
- is_constructible_v<T, const U&> is true,
- is_same_v<U, T> is false,
- if T is not *cv* bool, *converts-from-any-cvref*<T, basic_optional <U>> is false.

Effects: alloc is default-initialized. If rhs contains a value, the contained value is constructed by uses-allocator construction with allocator alloc and*rhs.
Postconditions: rhs.has_value() == this->has_value().
Throws: Any exception thrown by the selected constructor of T.
Remarks: If is_convertible_v<const AllocatorU&, Allocator> is false, this constructor is defined as deleted. If Allocator's default constructor and T's selected constructor are constexpr constructors, this constructor is a constexpr constructor.
The expression inside explicit is equivalent to:
!is_convertible_v<const U&, T>

template<class U , class AllocatorU>
constexpr explicit(*see below*) basic_optional(basic_optional<U, AllocatorU>&&);
Constraints:
- is_constructible_v<T, U> is true,

- is_same_v<U, T> is false,
- if T is not *cv* bool, *converts-from-any-cvref*<T, basic_optional <U, AllocatorU>> is false.

Effects: alloc is default-initialized. If rhs contains a value, the contained value is constructed by uses-allocator construction with allocator alloc and std :: move(*rhs). rhs.has_value() is unchanged.
Postconditions: rhs.has_value() == this->has_value().
Throws: Any exception thrown by the selected constructor of T.
Remarks: If is_convertible_v<const AllocatorU&, Allocator> is false, this constructor is defined as deleted. If Allocator's default constructor and T's selected constructor are constexpr constructors, this constructor is a constexpr constructor.

The expression inside explicit is equivalent to:

!is_convertible_v<U, T>

```
template<class U>
constexpr explicit(see below) basic_optional(const optional <U>&);
```

Constraints:
- is_constructible_v<T, const U&> is true,
- if T is not *cv* bool, *converts-from-any-cvref*<T, optional <U>> is false.

Effects: alloc is default-initialized. If rhs contains a value, the contained value is constructed by uses-allocator construction with allocator alloc and*rhs.
Postconditions: rhs.has_value() == this->has_value().
Throws: Any exception thrown by the selected constructor of T.
Remarks: If Allocator's default constructor and T's selected constructor are constexpr constructors, this constructor is a constexpr constructor.

The expression inside explicit is equivalent to:

!is_convertible_v<const U&, T>

```
template<class U>
constexpr explicit(see below) basic_optional(optional<U>&&);
```

Constraints:
- is_constructible_v<T, U> is true,
- if T is not *cv* bool, *converts-from-any-cvref*<T, optional <U>> is false.

Effects: alloc is default-initialized. If rhs contains a value, the contained value is constructed by uses-allocator construction with allocator alloc and std :: move(*rhs). rhs.has_value() is unchanged.
Postconditions: rhs.has_value() == this->has_value().
Throws: Any exception thrown by the selected constructor of T.
Remarks: If Allocator's default constructor and T's selected constructor are constexpr constructors, this constructor is a constexpr constructor.

The expression inside explicit is equivalent to:

!is_convertible_v<U, T>

```
constexpr basic_optional(allocator_arg_t, const allocator_type& a) noexcept;
constexpr basic_optional(allocator_arg_t, const allocator_type& a, nullopt_t) noexcept;
constexpr basic_optional(allocator_arg_t, const allocator_type& a, const basic_optional&);
constexpr basic_optional(allocator_arg_t, const allocator_type& a, basic_optional&&);
template<class... Args>
constexpr basic_optional(allocator_arg_t, const allocator_type& a, in_place_t, Args&&...);
template<class U, class... Args>
constexpr basic_optional(allocator_arg_t, const allocator_type& a, in_place_t, initializer_list<U>, Args&&...);
template<class U = T>
constexpr basic_optional(allocator_arg_t, const allocator_type& a, U&&);
template<class AllocatorU>
constexpr basic_optional(allocator_arg_t, const allocator_type& a, const basic_optional <T, AllocatorU>&);
template<class AllocatorU>
constexpr basic_optional(allocator_arg_t, const allocator_type& a, basic_optional<T, AllocatorU>&&);
template<class U , class AllocatorU>
constexpr basic_optional(allocator_arg_t, const allocator_type& a, const basic_optional <U, AllocatorU>&);
template<class U , class AllocatorU>
constexpr basic_optional(allocator_arg_t, const allocator_type& a, basic_optional<U, AllocatorU>&&);
template<class U>
constexpr basic_optional(allocator_arg_t, const allocator_type& a, const optional <U>&);
template<class U>
constexpr basic_optional(allocator_arg_t, const allocator_type& a, optional<U>&&);
```

Effects: Behaves the same as non allocator extended version of the constructor except it initializes alloc with the specified allocator before initializing the alternative, if any, by uses-allocator construction.

22.5.x.3 Destructor[basic.optional.dtor]

constexpr ~basic_optional();
Effects: If is_trivially_destructible_v<T> != true and *this contains a value, calls
val->T::~T()
Remarks: If is_trivially_destructible_v<T> is true, then this destructor is trivial

22.5.x.4 Assignment [basic.optional.assign]

onstexpr basic_optional <T, Allocator>& operator=(nullopt_t) noexcept;
Effects: If *this contains a value, calls val->T :: ~T() to destroy the contained value; otherwise no effect
Postconditions: *this does not contain a value
Returns: *this

constexpr basic_optional <T, Allocator><T>& operator=(const basic_optional& rhs);
Effects: If allocator_traits<Allocator>::propagate_on_container_copy_assignment::value is true, sets alloc to rhs.alloc.
Then, see Table x

Table — basic_optional operator=(const basic_optional& rhs) effects

	***this contains a value**	***this does not contain a value**
rhs contains a value	assigns *rhs to the contained value	if uses_allocator_v<remove_cv_t<T>>, Allocator> is false or allocator_traits<Allocator>::is_always_equal::value is true, direct-non-list-initializes the contained value with *rhs. Otherwise, constructs the contained value by uses-allocator construction with allocator alloc and*rhs
rhs does not contain a value	Destroys the contained value by calling val->T::~T()	No additional effect

Postconditions: rhs.has_value() == this->has_value().
Returns: *this.
Remarks: If any exception is thrown, the result of the expression this->has_value() remains unchanged. If an exception is thrown during the call to T's selected copy constructor, no effect. If an exception is thrown during the call to T's copy assignment, the state of its contained value is as defined by the exception safety guarantee of T's copy assignment. This operator is defined as deleted unless is_copy_constructible_v<T> is true and is_copy_assignable_v<T> is true.
The assignment operator is trivial if :
- is_trivially_copy_constructible_v<T> && is_trivially_copy_assignable_v<T> && is_trivially_destructible_v<T> is true,
-
is_trivially_copy_constructible_v<Allocator> && is_trivially_copy_assignable_v<Allocator> && is_trivially_destructible_v<Allocator> is true, and
- allocator_traits<Allocator>::is_always_equal::value is true

constexpr basic_optional& operator=(basic_optional&& rhs) noexcept(*see below*);
Constraints: is_move_constructible_v<T> is true and is_move_assignable_v<T> is true.
Effects: If allocator_traits<Allocator>::propagate_on_container_move_assignment::value is true, sets alloc to rhs.alloc.
Then, see TableX. The result of the expression rhs.has_value() remains unchanged

Table — basic_optional operator=(basic_optional&& rhs) effects

	***this contains a value**	***this does not contain a value**
rhs contains a value	assigns std::move(*rhs) to the contained value	if uses_allocator_v<remove_cv_t<T>>, Allocator> is false or allocator_traits<Allocator>::is_always_equal::value is true or or allocator_traits<Allocator>::propagate_on_container_move_assignment::value is true, direct-non-list-initializes the contained value with std::move(*rhs). Otherwise, constructs the contained value by uses-allocator construction with allocator alloc and*rhs
rhs does not contain a value	Destroys the contained value by calling val->T::~T()	No additional effect

Postconditions: rhs.has_value() == this->has_value().

Returns: *this.
Remarks: The exception specification is equivalent to:
is_nothrow_move_assignable_v<T> && is_nothrow_move_constructible_v<T>&& (!uses_allocator_v<remove_cv_t<T>>, Allocator> || allocator_traits<Allocator>::is_always_equal::value || allocator_traits<Allocator>::propagate_on_container_move_assignment::value)
If any exception is thrown, the result of the expression this->has_value() remains unchanged.
If an exception is thrown during the call to T's move constructor, the state of *rhs.val is determined by the exception safety guarantee of T's selected move constructor. If an exception is thrown during the call to T's move assignment, the state of *val and *rhs.val is determined by the exception safety guarantee of T's move assignment.
If

- is_trivially_move_constructible_v<T> && is_trivially_move_assignable_v<T> && is_trivially_destructible_v<T> is true, and

- is_trivially_move_constructible_v<Allocator> && is_trivially_move_assignable_v<Allocator> && is_trivially_destructible_v<Allocator> is true
- allocator_traits<Allocator>::is_always_equal::value is true or allocator_traits<Allocator>::propagate_on_container_move_assignment::value is true

this assignment operator is trivial.

```
template<class AllocatorU>
constexpr basic_optional <T, Allocator><T>& operator=(const basic_optional<T, AllocatorU>& rhs);
```
Constraints: is_copy_constructible_v<T> is true and is_copy_assignable_v<T> is true.
Effects: If allocator_traits<Allocator>::propagate_on_container_copy_assignment::value is true, sets alloc to rhs.alloc.
Then, see Table x

Table — basic_optional operator=(const basic_optional<T, AllocatorU>& rhs) effects

	***this contains a value**	***this does not contain a value**
rhs contains a value	assigns *rhs to the contained value	if uses_allocator_v<remove_cv_t<T>>, Allocator> is false or allocator_traits<Allocator>::is_always_equal::value is true, direct-non-list-initializes the contained value with *rhs. Otherwise, constructs the contained value by uses-allocator construction with allocator alloc and*rhs
rhs does not contain a value	Destroys the contained value by calling val->T::~T()	No additional effect

Postconditions: rhs.has_value() == this->has_value().
Returns: *this.
Remarks: If any exception is thrown, the result of the expression this->has_value() remains unchanged. If an exception is thrown during the call to T's selected copy constructor, no effect. If an exception is thrown during the call to T's copy assignment, the state of its contained value is as defined by the exception safety guarantee of T's copy assignment.
This operator is defined as deleted if is_convertible_v<const AllocatorU&, Allocator> is false.

```
template<class AllocatorU>
constexpr basic_optional& operator=(basic_optional<T, AllocatorU>&& rhs) noexcept(see below);
```
Constraints: is_move_constructible_v<T> is true and is_move_assignable_v<T> is true.
Effects: If allocator_traits<Allocator>::propagate_on_container_move_assignment::value is true, sets alloc to rhs.alloc.
Then, see TableX. The result of the expression rhs.has_value() remains unchanged

Table — basic_optional operator=(basic_optional<T, AllocatorU>&& rhs) effects

	***this contains a value**	***this does not contain a value**
rhs contains a value	assigns std::move(*rhs) to the contained value	if uses_allocator_v<remove_cv_t<T>>, Allocator> is false or allocator_traits<Allocator>::is_always_equal::value is true or or allocator_traits<Allocator>::propagate_on_container_move_assignment::value is true, direct-non-list-initializes the contained value with std::move(*rhs). Otherwise, constructs the contained value by uses-allocator construction with allocator alloc and*rhs
rhs does not contain a value	Destroys the contained value by calling val->T::~T()	No additional effect

Postconditions: rhs.has_value() == this->has_value().
Returns: *this.

Remarks: The exception specification is equivalent to:
is_nothrow_move_assignable_v<T> && is_nothrow_move_constructible_v<T>&& (!uses_allocator_v<remove_cv_t<T>>, Allocator> || allocator_traits<Allocator>::is_always_equal::value || allocator_traits<Allocator>::propagate_on_container_move_assignment::value)
If any exception is thrown, the result of the expression this->has_value() remains unchanged.
If an exception is thrown during the call to T's move constructor, the state of *rhs.val is determined by the exception safety guarantee of T's selected move constructor. If an exception is thrown during the call to T's move assignment, the state of *val and *rhs.val is determined by the exception safety guarantee of T's move assignment.
This operator is defined as deleted if is_convertible_v<const AllocatorU&, Allocator> is false.

template<class U = T> constexpr basic_optional <T>& operator=(U&& v).
Effects: If *this contains a value, assigns std :: forward<U>(v) to the contained value; otherwise constructs the contained value by uses-allocator construction with allocator alloc and std::forward<U>(v).
Postconditions: *this contains a value.
Returns: *this
Remarks: If any exception is thrown, the result of the expression this->has_value() remains unchanged. If an exception is thrown during the call to T's constructor, the state of v is determined by the exception safety guarantee of T's selected constructor. If an exception is thrown during the call to T's assignment, the state of *val and v is determined by the exception safety guarantee of T's assignment.

template<class U, class AllocatorU> constexpr basic_optional <T>& operator=(const basic_optional <U, AllocatorU>& rhs);
Constraints:
- is_constructible_v<T, const U&> is true,
- is_assignable_v<T&, const U&> is true,
- is_same_v<U, T> is false,
- *converts-from-any-cvref*<T, basic_optional<U, AllocatorU>> is false,
- is_assignable_v<T&, basic_optional<U, AllocatorU>&> is false,
- is_assignable_v<T&, basic_optional<U, AllocatorU>&&> is false,
- is_assignable_v<T&, const basic_optional<U, AllocatorU>&> is false, and
- is_assignable_v<T&, const basic_optional<U, AllocatorU>&&> is false.

Effects: See Table

Table optional ::operator=(const basic_optional<U, AllocatorU>&) effects

Table — basic_optional operator=(const basic_optional<U, AllocatorU>&) effects

	***this contains a value**	***this does not contain a value**
rhs contains a value	assigns *rhs to the contained value	constructs the contained value by uses-allocator construction with allocator alloc and *rhs
rhs does not contain a value	Destroys the contained value by calling val->T::~T()	No additional effect

Postconditions: rhs.has_value() == this->has_value().
Returns: *this.
Remarks: If is_convertible_v<const AllocatorU&, Allocator> is false, this operator is defined as deleted. If any exception is thrown, the result of the expression this->has_value() remains unchanged. If an exception is thrown during the call to T's constructor, the state of *rhs.val is determined by the exception safety guarantee of T's selected constructor If an exception is thrown during the call to T's assignment, the state of *val and *rhs.val is determined by the exception safety guarantee of T's assignment.

template<class U, class AllocatorU> constexpr optional<T>& operator=(basic_optional <U, AllocatorU>&& rhs);
Constraints:
- is_constructible_v<T, U> is true,
- is_assignable_v<T&, U> is true,
- is_same_v<U, T> is false,
- *converts-from-any-cvref*<T, basic_optional <U, AllocatorU>> is false,
- is_assignable_v<T&, basic_optional <U, AllocatorU>&> is false,
- is_assignable_v<T&, basic_optional <U, AllocatorU>&&> is false,
- is_assignable_v<T&, const basic_optional <U, AllocatorU>&> is false, and
- is_assignable_v<T&, const basic_optional <U, AllocatorU>&&> is false

Effects: See Table. The result of the expression rhs.has_value() remains unchanged

Table — basic_optional operator=(basic_optional <U, AllocatorU>&&) effects

	*this contains a value	*this does not contain a value
rhs contains a value	assigns std::move(*rhs) to the contained value	constructs the contained value by uses-allocator construction with allocator alloc and std::move(*rhs)
rhs does not contain a value	Destroys the contained value by calling val->T::~T()	No additional effect

Postconditions: rhs.has_value() == this->has_value().
Returns: *this.
Remarks: If is_convertible_v<const AllocatorU&, Allocator> is false, this operator is defined as deleted. If any exception is thrown, the result of the expression this->has_value() remains unchanged. If an exception is thrown during the call to T's constructor, the state of *rhs.val is determined by the exception safety guarantee of T's selected constructor If an exception is thrown during the call to T's assignment, the state of *val and *rhs.val is determined by the exception safety guarantee of T's assignment.

template<class U> constexpr basic_optional <T>& operator=(const optional <U>& rhs);
Constraints:
- is_constructible_v<T, const U&> is true,
- is_assignable_v<T&, const U&> is true,
- *converts-from-any-cvref*<T, optional<U>> is false,
- is_assignable_v<T&, optional<U>&> is false,
- is_assignable_v<T&, optional<U>&&> is false,
- is_assignable_v<T&, const optional<U>&> is false, and
- is_assignable_v<T&, const optional<U>&&> is false.

Effects: See Table

Table — basic_optional operator=(const optional<U>&) effects

	*this contains a value	*this does not contain a value
rhs contains a value	assigns *rhs to the contained value	constructs the contained value by uses-allocator construction with allocator alloc and *rhs
rhs does not contain a value	Destroys the contained value by calling val->T::~T()	No additional effect

Postconditions: rhs.has_value() == this->has_value().
Returns: *this.
Remarks: If any exception is thrown, the result of the expression this->has_value() remains unchanged. If an exception is thrown during the call to T's constructor, the state of *rhs.val is determined by the exception safety guarantee of T's selected constructor If an exception is thrown during the call to T's assignment, the state of *val and *rhs.val is determined by the exception safety guarantee of T's assignment.

template<class U> constexpr optional<T>& operator=(optional<U>&& rhs);
Constraints:
- is_constructible_v<T, U> is true,
- is_assignable_v<T&, U> is true,
- *converts-from-any-cvref*<T, optional<U>> is false,
- is_assignable_v<T&, optional<U>&> is false,
- is_assignable_v<T&, optional<U>&&> is false,
- is_assignable_v<T&, const optional<U>&> is false, and
- is_assignable_v<T&, const optional<U>&&> is false

Effects: See Table The result of the expression rhs.has_value() remains unchanged

Table — basic_optional operator=(optional<U>&&) effects

	*this contains a value	*this does not contain a value
rhs contains a value	assigns std::move(*rhs) to the contained value	constructs the contained value by uses-allocator construction with allocator alloc and std::move(*rhs)
rhs does not contain a	Destroys the contained value by calling val->T::~T()	No additional effect

value		

Postconditions: rhs.has_value() == this->has_value().
Returns: *this.
Remarks: If any exception is thrown, the result of the expression this->has_value() remains unchanged. If an exception is thrown during the call to T's constructor, the state of *rhs.val is determined by the exception safety guarantee of T's selected constructor If an exception is thrown during the call to T's assignment, the state of *val and *rhs.val is determined by the exception safety guarantee of T's assignment.

```
template<class... Args> constexpr T& emplace(Args&&... args);
```
Mandates: is_constructible_v<T, Args...> is true
Effects: Calls *this = nullopt. Then constructs the contained value by uses-allocator construction with allocator alloc and std :: forward <Args>(args)….
Postconditions: *this contains a value.
Returns: A reference to the new contained value.
Throws: Any exception thrown by the selected constructor of T
Remarks: If an exception is thrown during the call to T's constructor, *this does not contain a value, and the previous *val (if any) has been destroyed.

```
template<class U, class... Args> constexpr T& emplace(initializer_list<U> il, Args&&... args);
```
Constraints: is_constructible_v<T, initializer_list<U>&, Args...> is true
Effects: Calls *this = nullopt. Then constructs the contained value by uses-allocator construction with allocator alloc and il, std :: forward<Args>(args)….
Postconditions: *this contains a value
Returns: A reference to the new contained value
Throws: Any exception thrown by the selected constructor of T
Remarks: If an exception is thrown during the call to T's constructor, *this does not contain a value, and the previous *val (if any) has been destroyed.

22.5.x.5 Swap [optional.swap]

```
constexpr void swap(basic_optional& rhs) noexcept(see below);
```

Mandates: is_move_constructible_v<T> is true
Preconditions: T meets the *Cpp17Swappable* requirements ([swappable.requirements])
Effects: If allocator_traits<Allocator>::propagate_on_container_swap::value is true, then Allocator shall meet the *Cpp17Swappable* requirements and the allocators of *this and res shall also be exchanged by calling swap as described in [swappable.requirements]. Otherwise, the allocators shall not be swapped, and the behavior is undefined unless *this.get_allocator() == rhs.get_allocator(). Then, see Tablex

Table — basic_optional swap(basic_optional<U>&) effects

	***this contains a value**	***this does not contain a value**
rhs contains a value	calls swap(*(*this), *rhs)	constructs the contained value in *this by uses-allocator construction with allocator alloc and std :: move(*rhs), followed by rhs.val->T :: ~T(); postcondition is that *this contains a value and rhs does not contain a value
rhs does not contain a value	constructs the contained value of rhs by uses-allocator construction with allocator alloc and std :: move(*(*this)), followed by val->T :: ~T(); postcondition is that *this does not contain a value and rhs contains a value	No additional effect

Throws: Any exceptions thrown by the operations in the relevant part of Table.
Remarks: The exception specification is equivalent to:
is_nothrow_move_constructible_v<T> && is_nothrow_swappable_v<T>
If any exception is thrown, the results of the expressions this->has_value() and rhs.has_value() remain unchanged. If an exception is thrown during the call to function swap, the state of *val and *rhs.val is determined by the exception safety guarantee of swap for lvalues of T. If an exception is thrown during the call to T's move constructor, the state of *val and *rhs.val is determined by the exception safety guarantee of T's selected move constructor.

22.5.x.6 Allocator [basic.optional.all]

```
allocator_type get_allocator() const;
```

Returns: A copy of the Allocator that was passed to the object's constructor or, if that allocator has been replaced, a copy of the most recent replacement.

22.5.x.7 Observers [basic.optional.observe]

```
constexpr const T* operator->() const noexcept;
constexpr T* operator->() noexcept;
```
Preconditions: *this contains a value
Returns: val.
Remarks: These functions are constexpr functions.

```
constexpr const T& operator*() const & noexcept;
constexpr T& operator*() & noexcept;
```
Preconditions: *this contains a value.
Returns: *val.
Remarks: These functions are constexpr functions.

```
constexpr T&& operator*() && noexcept;
constexpr const T&& operator*() const && noexcept;
```
Preconditions: *this contains a value.
Effects: Equivalent to: return std :: move(*val);
```
constexpr explicit operator bool() const noexcept;
```
Returns: true if and only if *this contains a value
Remarks: This function is a constexpr function.
```
constexpr bool has_value() const noexcept;
```
Returns: true if and only if *this contains a value.
Remarks: This function is a constexpr function.

```
constexpr const T& value() const &;
constexpr T& value() &;
```
Effects: Equivalent to:
```
    return has_value() ? *val : throw bad_optional_access();
```

```
constexpr T&& value() &&;
constexpr const T&& value() const &&;
```
Effects: Equivalent to:
```
return has_value() ? std::move(*val) : throw bad_optional_access();
```

```
template<class U> constexpr T value_or(U&& v) const &;
```
Mandates: is_copy_constructible_v<T> && is_convertible_v<U&&, T> is true
Effects: Equivalent to:
```
return has_value() ? **this : static_cast<T>(std::forward<U>(v));
```

```
template<class U> constexpr T value_or(U&& v) &&;
```
Mandates: is_move_constructible_v<T> && is_convertible_v<U&&, T> is true.
Effects: Equivalent to:
```
return has_value() ? std::move(**this) : static_cast<T>(std::forward<U>(v));
```

22.5.x.8 Monadic operations [basic.optional.monadic]

```
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) const &;
```
Let U be invoke_result_t<F, decltype(*val)>
Mandates: remove_cvref_t<U> is a specialization of basic_optional.
Effects: Equivalent to:
```
    if (*this) {
        return invoke(std::forward<F>(f), *val);
    } else {
        return remove_cvref_t<U>();
    }
```

```
template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &&;
```
Let U be invoke_result_t<F, decltype(std :: move(*val))>
Mandates: remove_cvref_t<U> is a specialization of basic_optional.
Effects: Equivalent to:

```
    if (*this) {
        return invoke(std::forward<F>(f), std::move(*val));
    } else {
        return remove_cvref_t<U>();
    }
```

```
template<class F> constexpr auto transform(F&& f) &;
template<class F> constexpr auto transform(F&& f) const &;
```
Let U be remove_cv_t<invoke_result_t<F, decltype(*val)>>
Mandates: U is a non-array object type other than in_place_t or nullopt_t. The declaration
```
        U u(invoke(std::forward<F>(f), *val));
```
is well-formed for some invented variable u.
[*Note 1*: There is no requirement that U is movable ([dcl.init.general]). — *end note*]
Returns: If *this contains a value, a basic_optional<U, Allocator> object whose contained value is direct-non-list-initialized with invoke(std::forward<F>(f), *val); otherwise, optional<U, Allocator>().

```
template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &&;
```
Let U be remove_cv_t<invoke_result_t<F, decltype(std :: move(*val))>>
Mandates: U is a non-array object type other than in_place_t or nullopt_t. The declaration
```
        U u(invoke(std::forward<F>(f), std::move(*val)));
```
is well-formed for some invented variable u.
[*Note 2*: There is no requirement that U is movable ([dcl.init.general]). — *end note*]
Returns: If *this contains a value, a basic_optional<U, Allocator> object whose contained value is direct-non-list-initialized with invoke(std::forward<F>(f), std::move(*val)); otherwise, basic_optional<U, Allocator>().

```
template<class F> constexpr optional or_else(F&& f) const &;
```
Constraints: F models invocable<> and T models copy_constructible
Mandates: is_same_v<remove_cvref_t<invoke_result_t<F>>, basic_optional> is true.
Effects: Equivalent to:
```
    if (*this) {
        return *this;
    } else {
        return std::forward<F>(f)();
    }
```

```
template<class F> constexpr optional or_else(F&& f) &&;
```
Constraints: F models invocable<> and T models move_constructible.
Mandates: is_same_v<remove_cvref_t<invoke_result_t<F>>, basic_optional> is true.
Effects: Equivalent to:
```
    if (*this) {
        return std::move(*this);
    } else {
        return std::forward<F>(f)();
    }
```

22.5.x.9 Modifiers [basic.optional.mod]

```
constexpr void reset() noexcept;
```
Effects: If *this contains a value, calls val->T :: ~T() to destroy the contained value; otherwise no effect.
Postconditions: *this does not contain a value.

Modify 22.5.6 Relational operators [optional.relops]

```
template<class T, class U> constexpr bool operator==(const optional<T>& x, const optional<U>& y);
template<class T, AllocatorT , class U, AllocatorU> constexpr bool operator==(const basic_optional<T, AllocatorT>& x, const basic_optional<U, AllocatorU>& y);
```
Mandates: The expression *x == *y is well-formed and its result is convertible to bool.
…
```
template<class T, class U> constexpr bool operator!=(const optional<T>& x, const optional<U>& y);
template<class T, AllocatorT , class U, AllocatorU> constexpr bool operator!=(const basic_optional<T, AllocatorT>& x, const basic_optional<U, AllocatorU>& y);
```
Mandates: The expression *x != *y is well-formed and its result is convertible to bool
…
```
template<class T, class U> constexpr bool operator<(const optional<T>& x, const optional<U>& y);
```

```
template<class T, AllocatorT , class U, AllocatorU> constexpr bool operator<(const basic_optional<T, AllocatorT>& x, const basic_optional<U, AllocatorU>& y);
```
Mandates: *x < *y is well-formed and its result is convertible to bool
…

```
template<class T, class U> constexpr bool operator>(const optional<T>& x, const optional<U>& y);
template<class T, AllocatorT , class U, AllocatorU> constexpr bool operator>(const basic_optional<T, AllocatorT>& x, const basic_optional<U, AllocatorU>& y);
```
Mandates: The expression *x > *y is well-formed and its result is convertible to bool.

…
```
template<class T, class U> constexpr bool operator<=(const optional<T>& x, const optional<U>& y);
template<class T, AllocatorT , class U, AllocatorU> constexpr bool operator<=(const basic_optional<T, AllocatorT>& x, const basic_optional<U, AllocatorU>& y);
```
Mandates: The expression *x <= *y is well-formed and its result is convertible to bool.

…
```
template<class T, class U> constexpr bool operator>=(const optional<T>& x, const optional<U>& y);
template<class T, AllocatorT , class U, AllocatorU> constexpr bool operator>=(const basic_optional<T, AllocatorT>& x, const basic_optional<U, AllocatorU>& y);
```
…

```
template<class T, three_way_comparable_with<T> U>
  constexpr compare_three_way_result_t<T, U>
    operator<=>(const optional<T>& x, const optional<U>& y);
template<class T, AllocatorT , three_way_comparable_with<T> U AllocatorU>
constexpr compare_three_way_result_t<T, U>
    operator<=>(const basic_optional<T, AllocatorT>& x, const basic_optional<U, AllocatorU>& y);
```
Returns: If x && y, *x <=> *y; otherwise x.has_value() <=> y.has_value()

Modify 22.5.7 Comparison with nullopt

[optional.nullops]

```
template<class T> constexpr bool operator==(const optional<T>& x, nullopt_t) noexcept;
template<class T, class Allocator> constexpr bool operator==(const basic_optional<T, Allocator>& x, nullopt_t) noexcept;
```
Returns: !x.

```
template<class T> constexpr strong_ordering operator<=>(const optional<T>& x, nullopt_t) noexcept;
template<class T, class Allocator> constexpr strong_ordering operator<=>(const basic_optional<T, Allocator>& x, nullopt_t) noexcept;
```
Returns: x.has_value() <=> false

Modify 22.5.8 Comparison with T [optional.comp.with.t]

```
template<class T, class U> constexpr bool operator==(const optional<T>& x, const U& v);
template<class T, class Allocator, class U> constexpr bool operator==(const basic_optional<T, Allocator>& x, const U& v);
```
Mandates: The expression *x == v is well-formed and its result is convertible to bool.
…
```
template<class T, class U> constexpr bool operator==(const T& v, const optional<U>& x);
template<class T, class U, class Allocator,> constexpr bool operator==(const T& v, const basic_optional<U, Allocator>& x);
```
Mandates: The expression v == *x is well-formed and its result is convertible to bool.
…

```
template<class T, class U> constexpr bool operator!=(const optional<T>& x, const U& v);
template<class T, class Allocator, class U> constexpr bool operator!=(const basic_optional<T, Allocator>& x, const U& v);
```
Mandates: The expression *x != v is well-formed and its result is convertible to bool
…

```
template<class T, class U> constexpr bool operator!=(const T& v, const optional<U>& x);
emplate<class T, class U, class Allocator,> constexpr bool operator!=(const T& v, const basic_optional<U, Allocator>& x);
```
Mandates: The expression v != *x is well-formed and its result is convertible to bool.
…

```
template<class T, class U> constexpr bool operator<(const optional<T>& x, const U& v);
template<class T, class Allocator, class U> constexpr bool operator<(const basic_optional<T, Allocator>& x, const U& v);
```

Mandates: The expression *x < v is well-formed and its result is convertible to bool.
…

template<class T, class U> constexpr bool operator<(const T& v, const optional<U>& x);
template<class T, class U, class Allocator,> constexpr bool operato<(const T& v, const basic_optional<U, Allocator>& x);
Mandates: The expression v < *x is well-formed and its result is convertible to bool.
…

template<class T, class U> constexpr bool operator>(const optional<T>& x, const U& v);
template<class T, class Allocator, class U> constexpr bool operator>(const basic_optional<T, Allocator>& x, const U& v);
Mandates: The expression *x > v is well-formed and its result is convertible to bool.
…
template<class T, class U> constexpr bool operator>(const T& v, const optional<U>& x);
template<class T, class U, class Allocator,> constexpr bool operator>(const T& v, const basic_optional<U, Allocator>& x);
Mandates: The expression v > *x is well-formed and its result is convertible to bool.
…

template<class T, class U> constexpr bool operator<=(const optional<T>& x, const U& v);
template<class T, class Allocator, class U> constexpr bool operator<=(const basic_optional<T, Allocator>& x, const U& v);
Mandates: The expression *x <= v is well-formed and its result is convertible to bool
…
template<class T, class U> constexpr bool operator<=(const T& v, const optional<U>& x);
template<class T, class U, class Allocator,> constexpr bool operator<=(const T& v, const basic_optional<U, Allocator>& x);
Mandates: The expression v <= *x is well-formed and its result is convertible to bool
…

template<class T, class U> constexpr bool operator>=(const optional<T>& x, const U& v);
template<class T, class Allocator, class U> constexpr bool operator>=(const basic_optional<T, Allocator>& x, const U& v);
Mandates: The expression *x >= v is well-formed and its result is convertible to bool.
…

template<class T, class U> constexpr bool operator>=(const T& v, const optional<U>& x);
template<class T, class U, class Allocator,> constexpr bool operator>=(const T& v, const basic_optional<U, Allocator>& x);
Mandates: The expression v >= *x is well-formed and its result is convertible to bool.
…

```
template<class T, class U>
  requires (!is-derived-from-optional<U>) && three_way_comparable_with<T, U>
constexpr compare_three_way_result_t<T, U>
  operator<=>(const optional<T>& x, const U& v);
template<class T, class Allocator, class U>
  requires (!is-derived-from-optional<U>) && three_way_comparable_with<T, U>
constexpr compare_three_way_result_t<T, U>
  operator<=>(const optional<T, Allocator>& x, const U& v);
```

Effects: Equivalent to: return x.has_value() ? *x <=> v : strong_ordering :: less;

Modify 22.5.10 Hash support [optional.hash]

template<class T> struct hash<optional<T>>;
The specialization hash<optional<T>> is enabled ([unord.hash]) if and only if hash<remove_const_t<T>> is enabled. When enabled, for an object o of type optional<T>, if o.has_value() == true, then hash<optional<T>>()(o) evaluates to the same value as hash<remove_const_t<T>>()(*o); otherwise it evaluates to an unspecified value. The member functions are not guaranteed to be noexcept.

template<class T, class Allocator> struct hash<basic_optional<T, Allocator>>;
The specialization hash<basic_optional<T, Allocator>> is enabled ([unord.hash]) if and only if hash<remove_const_t<T>> is enabled. When enabled, for an object o of type basic_optional<T, Allocator>, if o.has_value() == true, then hash<basic_optional<T, Allocator>>()(o) evaluates to the same value as hash<remove_const_t<T>>()(*o); otherwise it evaluates to an unspecified value. The member functions are not guaranteed to be noexcept.